

Tentu, mari kita lanjut ke file **Distortion.cs**.

File ini menangani Transformasi Geometri, khususnya efek Distorsi (Shearing). Efek ini membuat gambar terlihat "miring" atau "tertarik" ke satu arah, seolah-olah gambar itu terbuat dari karet yang ditarik ujungnya.

1. Gambaran Besar: Apa itu Shearing?

Shearing adalah menggeser posisi pixel berdasarkan koordinatnya.

- **Shear X**: Semakin ke bawah posisinya (Y besar), semakin digeser ke kanan/kiri. (Bayangkan menumpuk buku lalu didorong sampingnya).
 - **Shear Y**: Semakin ke kanan posisinya (X besar), semakin digeser ke atas/bawah.
-

2. Penjelasan Kode Per Bagian

A. Bagian UI (Input User) - RequestDistortion

Tidak seperti Canny yang otomatis, Distorsi butuh input angka dari user: "Mau dimiringkan seberapa derajat?". Kode ini membuat **Form Pop-up** secara manual (tanpa Designer).

C#

```
public Bitmap RequestDistortion(Bitmap sourceImage)
{
    // Membuat Form baru (Jendela dialog kecil)
    using (Form inputForm = new Form())
    {
        // ... (Setting judul, ukuran, tombol close) ...

        // Membuat kotak input (TextBox) untuk nilai X dan Y
        TextBox txtX = new TextBox() { ... Text = "0.5" }; // Default 0.5
        TextBox txtY = new TextBox() { ... Text = "0" };

        // Tombol OK diklik
        btnOk.Click += (s, e) => {
            // Coba ubah teks jadi angka (float)
            if (float.TryParse(txtX.Text, out float sx) &&
                float.TryParse(txtY.Text, out float sy))
            {
```

```

        // Panggil logika utama
        result = ProcessDistortion(sourceImage, sx, sy);
    }
};

inputForm.ShowDialog(); // Tampilkan jendela
}
return result;
}

```

Poin UAS:

- Kode ini menunjukkan cara membuat UI **secara dinamis (lewat kodingan)**, bukan lewat *drag-and-drop* Visual Studio. Ini berguna untuk bikin dialog input sederhana yang cepat.

B. Logika Utama - ProcessDistortion (BAGIAN PENTING)

Di sini kita menggunakan Matematika Matriks.

Langkah 1: Siapkan Matriks

C#

```

Matrix matrix = new Matrix();
matrix.Shear(shearX, shearY); // Masukkan rumus geser miring

```

Langkah 2: Hitung Ukuran Kanvas Baru (Bounding Box)

Ini bagian paling tricky. Kalau gambar kotak ditarik miring, dia jadi jajar genjang. Ukurannya pasti melebar atau meninggi. Kalau kita pakai ukuran lama, gambarnya kepotong!

C#

```

// Buat kotak bayangan seukuran gambar asli
GraphicsPath path = new GraphicsPath();
path.AddRectangle(new RectangleF(0f, 0f, sourceImage.Width, sourceImage.Height));

// Terapkan matriks miring tadi ke kotak bayangan
path.Transform(matrix);

```

```
// Ukur seberapa besar kotak bayangan SEKARANG (setelah dimiringkan)
RectangleF bounds = path.GetBounds();
```

Langkah 3: Persiapan Kanvas & Koordinat

C#

```
// Buat kertas baru sesuai ukuran hasil hitungan tadi
Bitmap newImage = new Bitmap((int)bounds.Width, (int)bounds.Height);

using (Graphics g = Graphics.FromImage(newImage))
{
    g.Clear(Color.Black); // Cat background hitam
    // Biar hasil miringnya halus (tidak bergerigi)
    g.InterpolationMode = InterpolationMode.HighQualityBicubic;

    // --- KOREKSI POSISI (PENTING) ---
    Matrix m = new Matrix();

    // Geser titik nol!
    // Kalau gambar miring ke kiri, koordinatnya bisa jadi minus (misal -50).
    // Kita harus geser balik (+50) biar masuk ke tengah kanvas.
    // bounds.Left biasanya bernilai negatif jika di-shear ke kiri.
    m.Translate(-bounds.Left, -bounds.Top);

    // Terapkan miringnya lagi
    m.Shear(shearX, shearY);

    // Pasang matriks "Geser + Miring" ini ke alat lukis (Graphics)
    g.Transform = m;

    // Gambar!
    g.DrawImage(sourceImage, 0, 0);
}
```

1. Pertanyaan Konsep (Bounding Box)

Dosen: "Kenapa kamu repot-repot pakai `GraphicsPath` dan `GetBounds`? Kenapa tidak langsung `new Bitmap(width, height)` saja?"

Jawaban Anda:

"Karena saat gambar didistorsi (miring), bentuknya berubah jadi jajar genjang, Pak.
Lebar dan tingginya pasti bertambah. Kalau saya pakai ukuran asli, ujung-ujung gambar yang miring itu akan terpotong (keluar kanvas).
`GetBounds()` gunanya untuk menghitung luas area kotak baru yang pas untuk menampung jajar genjang tersebut."

2. Pertanyaan Koordinat (Translate)

Dosen: "Jelaskan baris `m.Translate(-bounds.Left, -bounds.Top)`. Kenapa dikasih minus?"

Jawaban Anda:

"Saat kita melakukan Shearing, titik koordinat gambar bisa bergeser ke area negatif (Kuadran kiri/atas).
Contoh: Jika saya miringkan ke kiri, titik pojok gambar bisa pindah ke $X = -100$. Kalau langsung digambar, bagian itu hilang.
`Translate(-bounds.Left)` fungsinya menggeser sistem koordinat kembali ke titik 0,0 agar seluruh gambar masuk ke dalam kanvas."

3. Pertanyaan Kualitas Gambar

Dosen: "Gambar hasil miringnya kok bisa halus? Padahal pixel kan kotak-kotak?"

Jawaban Anda:

"Karena saya mengaktifkan `InterpolationMode.HighQualityBicubic`, Pak.
Ini memerintahkan komputer untuk menghitung rata-rata warna pixel baru (interpolasi) saat gambar ditarik, bukan sekadar memindahkan titiknya secara kasar (Nearest Neighbor) yang bikin gambar pecah/bergerigi."

4. Pertanyaan Matriks

Dosen: "Apa yang sebenarnya dilakukan `matrix.Shear(0.5, 0)`?"

Jawaban Anda:

"Itu akan menggeser gambar secara Horizontal.
Setiap pixel akan digeser ke kanan sebesar $0.5 * Y$.
Artinya: Bagian atas gambar ($Y=0$) tidak geser. Bagian bawah gambar (Y besar)

akan geser jauh ke kanan. Efeknya gambar jadi miring seperti huruf Italic."

Apakah penjelasan ini cukup jelas? Kode Distortion.cs ini pendek tapi logika matematikanya (Matriks & Koordinat) cukup dalam.